

# SDD-Crash-Course: Complete End-to-End Engineering Course

Mastering Context-Driven Development (Conductor), Delta Contracts (OpenSpec), and Spec-as-SSOT (SPEC.md)

## 1. Conductor (Context & TDD Engine)

**Unit of Work:** Track (`conductor/tracks/<id>/`)

- Persistent repo constitution (`product.md`, `tech-stack.md`, `workflow.md`).
- Interactive 7-Section Track `spec.md` & phased `plan.md`.
- Strict **Red** → **Green** → **Refactor** TDD execution (`/conductor:implement`) with `git notes` & SHA checkpoints.

## 2. OpenSpec (Delta Contract Engine)

**Unit of Work:** Capability Delta

(`openspec/changes/<id>/`)

- Two-Plane isolation: `openspec/specs/` (Current Truth) vs. `openspec/changes/` (Proposed Delta).
- Normative RFC 2119 (`SHALL`, `MUST`) + 4-hashtag `####`  
`Scenario: (GIVEN/WHEN/THEN)`.
- Deterministic merge via `/opsx:verify` & `/opsx:archive`.

## 3. SPEC.md (Single Source of Truth)

**Unit of Work:** Whole-System Blueprint (`SPEC.md`)

- Canonical 8-Section architecture, schema, API, and deployment specification.
- Separates **Essential Domain Intent** from incidental implementation details.
- Verified by the **Clean-Room Rebuild Test** against read-only adversarial tests.

## 7-Module Step-by-Step Curriculum: Theory + 6 Executable Python Engineering Labs

- **Module 01:** Foundations, Software 3.0 Verification Loop, Conductor & OpenSpec Architecture
- **Module 02 (Lab 01):** Greenfield Rate Limiter Specs & ADRs
- **Module 03 (Lab 02):** Brownfield Lease Lock & Fencing Tokens
- **Module 04 (Lab 03):** Circuit Breaker TDD State Machine
- **Module 05 (Lab 04):** Token Bucket Drift & 5-Step Scorecard Refactoring
- **Module 06 (Lab 05):** KVCache SDD Code Review & Stacked PRs
- **Module 07 (Lab 06):** CoursePulse `SPEC.md` Clean-Room Rebuild

# From “Vibe Coding” to Agentic Engineering: Validation & Human Interaction

Why generation is free, human verification is the bottleneck, and agents need a short leash + immutable verifiers

## Anti-Pattern: The Vibe Coding Amnesia Loop

- 1. Silent Assumptions:** “Overeager junior” agent guesses ambiguous edge cases without asking the human.
- 2. 1,000-Line Bloat & Collateral Edits:** Agent over-engineers abstractions and mutates adjacent working code (e.g. FIFO → LRU).
- 3. Sycophantic Self-Testing:** When the same unconstrained prompt writes code and tests, it weakens assertions so broken code passes.
- 4. Unreviewable PR Slop:** 2,000-line diffs overwhelm human reviewers.

## Solution: 4 Human Gates & 4 Validation Layers

- 1. Think Before Coding:** Sandboxed `/conductor:newTrack` Q&A surfaces trade-offs before any code is written.
- 2. Spec-First Steering (`program.md` Pattern):** Human approves 50-line RFC 2119 + Gherkin `spec.md` before `/conductor:implement`.
- 3. Surgical Scope & Short Leash:** Lock `Out of Scope` + `tech-stack.md`; verify at small Phase Checkpoints & Stacked PR batches.
- 4. Immutable Verifiers (`chmod -w`):** 1:1 `test_reqXXXX_*` TDD + read-only adversarial tests (Karpathy `autoresearch` pattern).

## Karpathy’s Software 3.0 Law: Speed Up the Generation–Verification Loop (“Iron Man Suit, Not Unsupervised Robot”)

- **4 Behavioral Agent Rules (in `conductor/workflow.md`):** (1) **Think Before Coding** (ask, don’t guess) · (2) **Simplicity First** (minimal TDD code) · (3) **Surgical Changes** (honor `Out of Scope`) · (4) **Goal-Driven Execution** (loop against objective verifiers: `openspec validate --strict`, `test_reqXXXX_*`, and `chmod -w` Clean-Room Rebuild Tests).

# The 3 Levels of Spec-Driven Development Maturity

Choosing the right level of specification rigor for greenfield, brownfield, and full-system rebuilds

## Level 1: Spec-First

“Write Spec, Generate Code, Discard Spec”

- Developer writes a prompt/spec to generate initial feature code.
- Subsequent bugfixes edit source code directly without updating the spec.
- **Failure Mode:** Spec becomes stale documentation within 2 sprints; never use for long-lived services.

## Level 2: Spec-Anchored

“Living Specs & Bidirectional Delta Sync”

- Powered by **OpenSpec** ([openspec/specs/](#)) and **Conductor** ([conductor/](#)).
- Every PR proposes explicit [ADDED](#), [MODIFIED](#), [REMOVED](#), or [RENAMED](#) requirement deltas.
- **Sweet Spot:** Default choice for 90% of daily team feature & brownfield engineering.

## Level 3: Spec-as-Source (SSOT)

“Deterministic Clean-Room Regeneration”

- Powered by canonical 8-Section [SPEC.md](#) + **Adversarial Test Suite**.
- Captures all schemas, API contracts, state machines, and parameterized IaC.
- **Verification:** Delete [src/](#), rebuild in [/tmp/](#) from [SPEC.md](#), pass [chmod -w](#) tests.

## Practitioner Decision Rule: How They Interlock in One Repository

- **Macro System Blueprint** ([SPEC.md](#)): Defines the 8-section architecture, data models, and clean-room rebuild contract.
- **Capability Contracts** ([openspec/specs/<cap>/spec.md](#)): Tracks requirement deltas ([REQ-XXXX](#)) across pull requests.
- **Execution Engine** ([conductor/tracks/<id>/](#)): Enforces stack locks, Red-Green-Refactor TDD, [git notes](#), and logical [/conductor:revert](#).

# Conductor: Context-Driven Development & TDD Track Execution

“Measure Twice, Code Once” — Persistent repository context, Plan Mode sandboxing, and Git-aware execution

## Step 1: `/conductor:setup`

### One-Time Repo Constitution

- Scans greenfield or brownfield repo (respecting `.gitignore`).
- Generates `conductor/index.md`, `product.md`, `product-guidelines.md`, `tech-stack.md`, `workflow.md`, and `tracks.md`.

## Step 2: `/conductor:newTrack`

### Sandboxed Spec & Phased Plan

- Enforces `policies/conductor.toml` (blocks writes outside `conductor/`).
- Conducts 3–5 interactive Q&A turns.
- Emits 7-section `spec.md`, hierarchical `plan.md`, and `metadata.json`.

## Step 3: `/conductor:implement`

### Strict Red-Green-Refactor TDD

- Marks task `[~]`, writes failing `test_reqXXXX_*` first (**Red**).
- Writes minimal code (**Green**), checks `>80%` coverage (`CI=true`).
- Commits code, attaches `git notes`, records `[x]` `<7-char-sha>`.

## Day-2 Engineering Commands: Auditing, Principal-Engineer Review & Logical Git Revert

- `/conductor:status` — Audits `tracks.md` and `plan.md` state markers (`[ ]`, `[~]`, `[x]` `sha`, `[checkpoint: sha]`) against Git history.
- `/conductor:review` — Audits diff against `spec.md`, `tech-stack.md`, and `code_styleguides/`; appends auto-fixes to `## Phase: Review Fixes`.
- `/conductor:revert` — 4-Phase Git-aware logical rollback of an entire Track, Phase, or Task (reconciling implementation + plan commits).

# OpenSpec: Two-Plane Architecture & Deterministic Delta Algebra

Separating “Current System Truth” (`openspec/specs/`) from “Proposed Changes” (`openspec/changes/`)

## Plane 1: `openspec/specs/<capability>/spec.md`

### The Living System of Record ("The Elephant")

- Organized by capability domain (e.g. `rate-limiter`, `lease-manager`, `kv-cache`), never by sprint ticket.
- Every requirement uses `### Requirement: [REQ-XXXX] <Title> + uppercase RFC 2119` (`SHALL`, `MUST`, `MUST NOT`).
- Every testable behavior uses **4-hashtag** `#### Scenario:` with `GIVEN`, `WHEN`, `THEN`.

## Plane 2: `openspec/changes/<change-id>/`

### Isolated Change Bundle ("The Goldfish")

- `proposal.md`: Why (50–1000 chars), What Changes, Capabilities, Impact.
- `design.md`: Context, Goals/Non-Goals, Decisions, Risks + `ADR-XXXX.md`.
- `specs/<cap>/spec.md`: Delta sections (`ADDED`, `MODIFIED`, `REMOVED`, `RENAMED`).
- `tasks.md`: Checkboxes (`- [ ] 1.1 ...`) grouped into Stacked PR batches.

## The 99% / 1% CLI Rule & Deterministic 5-Stage Archive Merge (`specs-apply.js`)

- **1% Terminal CLI**: Run `openspec init` once and `openspec validate --all --strict` in CI. **99% Slash Skills**: `/opsx:explore` → `/opsx:propose` → `/opsx:apply` → `/opsx:verify` → `/opsx:archive`.
- **Archive Execution Order**: (1) Pre-Validate Cross-Section Conflicts → (2) `RENAMED` → (3) `REMOVED` → (4) `MODIFIED` (full replacement!) → (5) `ADDED`.

# Wiring Conductor + OpenSpec + [@writing-arch-specs](#)

How `conductor/workflow.md` invokes dedicated spec skills and aligns tracks with `openspec/specs/`

## 1. `conductor/workflow.md` & `index.md` Handshake

```
# conductor/index.md
- [Writing Arch Specs Skill](../agents/skills/writing-arch-specs/SKILL.md)
- [OpenSpec Canonical Specs](../openspec/specs/)

# conductor/workflow.md
1. Skill Activation: Read .agents/skills/writing-arch-specs/SKILL.md
2. Contract Alignment: Map track spec.md 1:1 to REQ-XXXX in openspec/specs/
3. TDD Loop: Write failing test_reqXXXX_* first (Red -> Green -> Refactor)
4. Guardrails: Think Before Coding, Simplicity First, Surgical Changes, Immutable Verifiers
```

## 2. The 7-Section Track `spec.md` Anatomy

- 1. Overview:** Problem statement, business goal, target service.
- 2. Architecture:** Component topology, concurrency & data flow.
- 3. Functional Requirements:** Traceable `REQ-XXXX` contracts.
- 4. Non-Functional Requirements:** Latency, memory, lock contention.
- 5. Acceptance Criteria:** Binary pass/fail scenarios.
- 6. Out of Scope:** Explicit deferred items (prevents scope creep!).
- 7. Verification Commands:** Exact non-interactive `CI=true` commands.

## Why Dual-Engine Beats Either Tool Alone

- **OpenSpec alone** gives you clean capability contracts (`openspec/specs/`) and delta merges, but lacks Conductor's repo-wide `tech-stack.md` lock, `git notes` audit trail, and 4-phase logical `/conductor:revert`.
- **Conductor alone** gives you disciplined TDD track execution, but without OpenSpec its track specs remain isolated per ticket instead of merging into a permanent capability tree. **Together, they form a closed loop.**

# Lab 01 Step 1: Auditing the Ambiguous Proposal & Interactive Q&A

Walking through `labs/lab_01_greenfield_proposals_to_specs/proposal.md` (Think Before Coding)

## Input: The Vague Product Memo (`proposal.md`)

"We need an in-memory API Rate Limiter in Python so noisy tenants stop overloading our backend endpoints. Each client key should have a configurable request limit per time window, and callers should know whether a request is allowed or when to retry. Please ship this fast!"

**Why Vibe Coding Fails Here:** A raw coding agent immediately guesses fixed-window vs. sliding-window, ignores clock drift in unit tests, and forgets thread synchronization!

## Step 1 Action: `/opsx:explore` & `/conductor:newTrack` Q&A

In Plan Mode, the agent pauses and asks **5 Clarifying Questions**:

**Q1 (Window Semantics):** Fixed counter or exact sliding-window timestamp log? → *Exact sliding-window log (prevents boundary bursts).*

**Q2 (Weighted Calls):** Single-token only or configurable `cost >= 1`? → *Support `cost: int = 1`.*

**Q3 (Rejection Metadata):** Return boolean or structured `RateLimitDecision`? → *Return `allowed, remaining, retry_after_seconds`.*

**Q4 (Test Determinism):** Injectable `clock: Callable[[], float]`? → *Yes.*

**Q5 (Concurrency):** Thread-safe `threading.RLock`? → *Yes.*

## Hands-On Terminal & Prompt Recipe for Lab 01 Step 1

- **Inspect Starting Proposal:** `cd labs/lab_01_greenfield_proposals_to_specs && cat proposal.md`
- **Slash Command:** Run `/opsx:explore` to surface the 5 architectural ambiguities, then run `/opsx:propose "add-distributed-rate-limiter"` and `/conductor:newTrack "Implement Sliding-Window Rate Limiter MVP"`.

# Lab 01 Step 2: Authoring the OpenSpec Capability Contract

Writing `expected_output/openspec/specs/rate-limiter/spec.md` with `REQ-0001..REQ-0006`

## Exact OpenSpec Contract Excerpt (REQ-0001 & REQ-0002)

```
### Requirement: [REQ-0001] Sliding-Window Request Allowance
The RateLimiter SHALL evaluate requests against a sliding window
of '[now - window_seconds, now]' and MUST admit requests when
'active_tokens + cost <= limit'.

#### Scenario: Admit request within sliding window quota
- **GIVEN** a RateLimiter with 'limit=3' and 'window_seconds=10.0'
- **WHEN** client '"tenant-a"' calls 'allow("tenant-a", cost=2)' at 't=0.0'
- **THEN** 'allowed' SHALL be 'True', 'remaining' SHALL be '1',
  and 'retry_after_seconds' SHALL be '0.0'.

### Requirement: [REQ-0002] Quota Exhaustion & Retry-After
When 'active_tokens + cost > limit', the RateLimiter MUST reject
the request without recording new timestamps and SHALL compute
exact 'retry_after_seconds' until sufficient tokens expire.
```

## Complete Requirement Matrix (REQ-0001..REQ-0006)

- **REQ-0001 (Allowance):** Sliding window `[now - W, now]` admits when `used + cost <= limit`.
- **REQ-0002 (Rejection & Retry-After):** Rejects when `used + cost > limit`; computes exact time until oldest timestamps expire.
- **REQ-0003 (Config & Input Validation):** Raises `ValueError` on `limit < 1`, `window_seconds <= 0`, empty `key`, or `cost < 1`.
- **REQ-0004 (Injectable Clock):** Accepts `clock: Callable[[], float]` for zero-sleep deterministic tests.
- **REQ-0005 (Thread Safety):** Guards per-key deque pruning and admission with `threading.RLock`.
- **REQ-0006 (Key Reset):** `reset(key)` clears state for `key` only.

## Structural Validation Gate Check in `self_diagnose.sh`

- Verify every requirement header matches `### Requirement: [REQ-000X]`, contains uppercase `SHALL/MUST`, and includes a 4-hashtag `#### Scenario: block` with `GIVEN`, `WHEN`, and `THEN`.

# Lab 01 Step 3: Recording Architecture Decision Records (ADRs)

Evaluating storage and window trade-offs in [ADR-0001-storage-engine.md](#) and [ADR-0002-sync-protocol.md](#)

## ADR-0001: Storage Engine Selection

**Context:** Need sub-millisecond per-key timestamp pruning with zero external infra dependencies for MVP.

### Options Evaluated:

1. **In-Memory `dict[str, deque]` + `RLock` (ACCEPTED):**  $O(1)$  amortized popleft expiration, zero network hop, deterministic unit testing.
2. **Redis Sorted Set (`ZREMRANGEBYSCORE`) (DEFERRED):** Needed only for multi-node horizontal scaling; adds network/serialization overhead.
3. **SQLite / PostgreSQL Table (REJECTED):** Disk I/O & row lock contention violate  $< 1\text{ms}$  P99 latency budget.

## ADR-0002: Window Synchronization Protocol

**Context:** Must prevent  $2\times$  boundary traffic spikes at window edges while providing exact `retry_after_seconds`.

### Options Evaluated:

1. **Exact Sliding-Window Log (ACCEPTED):** Stores `(timestamp, cost)` entries in a monotonic deque; eliminates boundary bursts and computes exact `retry_after_seconds`.
2. **Fixed-Window Counter (REJECTED):** Permits  $2\times$  burst at window boundary ( $t = W - \epsilon$  and  $t = W + \epsilon$ ).
3. **Approximate Sliding Counter (REJECTED):** Cannot compute exact per-request `retry_after_seconds`.

## Why ADRs Are Mandatory in Spec-Driven Development

- Specifications (`spec.md`) capture **WHAT** the system must do; Architecture Decision Records (`ADR-XXXX.md`) capture **WHY** rejected alternatives were ruled out—preventing future AI sessions from “refactoring” an exact sliding-window deque back into a buggy fixed-window counter!

# Lab 01 Step 4: Authoring the 7-Section Conductor Track `spec.md` & `plan.md`

Bridging OpenSpec contracts into Conductor's phased Red-Green-Refactor execution plan

## 7-Section Conductor Track `spec.md` Structure

```
## 1. Overview
Implement thread-safe SlidingWindowRateLimiter satisfying REQ-0001..0006.
## 2. Architecture
Per-key collections.deque[(float, int)] guarded by threading.RLock.
## 3. Functional Requirements
Maps 1:1 to REQ-0001..REQ-0006 in openspec/specs/rate-limiter/spec.md.
## 4. Non-Functional Requirements
O(1) amortized pruning; zero external runtime packages.
## 5. Acceptance Criteria
All 6 unit tests pass with 100% requirement traceability.
## 6. Out of Scope
Distributed Redis cluster sync; HTTP middleware wrapper.
## 7. Verification Commands

CI=true python3 -m unittest discover -s expected_output -p "test_*.py" -v
```

## Phased Conductor `plan.md` with Checkpoints

```
## Phase 1: Core Data Model & Validation [checkpoint: a1b2c3d]
- [x] Task 1.1: Define RateLimitDecision & validate inputs (REQ-0003) 11a2b3c
- [x] Task 1.2: Inject deterministic clock callable (REQ-0004) 22b3c4d
- [x] Task: Conductor - User Manual Verification 'Phase 1'

## Phase 2: Sliding Window & Retry-After [checkpoint: d4e5f6a]
- [x] Task 2.1: Implement deque pruning & allow() (REQ-0001) 33c4d5e
- [x] Task 2.2: Compute exact retry_after_seconds (REQ-0002) 44d5e6f
- [x] Task: Conductor - User Manual Verification 'Phase 2'

## Phase 3: Concurrency & Reset [checkpoint: 77a8b9c]

- [x] Task 3.1: Add RLock & reset(key) (REQ-0005, REQ-0006) 55e6f7a
```

## Human-in-the-Loop Gate Before Code Generation

- Notice Section 6 (`Out of Scope`): explicitly excluding Redis and HTTP middleware prevents the agent from writing 800 lines of unrequested boilerplate. Once you approve `spec.md` and `plan.md`, you trigger `/conductor:implement`.

# Lab 01 Step 5: Implementing `ratelimiter.py` & Running `./self_diagnose.sh`

Inspecting the reference implementation and verifying all 6 requirement-traceable unit tests

## Reference Implementation (`expected_output/ratelimiter.py`)

```
def allow(self, key: str, cost: int = 1) -> RateLimitDecision:
    if not key or not isinstance(key, str): raise ValueError("Invalid key")
    if cost < 1 or cost > self.limit: raise ValueError("Invalid cost")
    with self._lock:
        now = float(self._clock())
        cutoff = now - self.window_seconds
        q = self._events.setdefault(key, deque())
        while q and q[0][0] <= cutoff:
            q.popleft()
        used = sum(c for _, c in q)
        if used + cost <= self.limit:
            q.append((now, cost))
            return RateLimitDecision(True, self.limit - (used + cost), 0.0)
        needed = (used + cost) - self.limit
        freed = 0
        for ts, c in q:
            freed += c
            if freed >= needed:
                return RateLimitDecision(False, max(0, self.limit - used),
                                         max(0.0, (ts + self.window_seconds) - now))
```

## Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_01_greenfield_proposals_to_specs/self_diagnose.sh
=== Lab 01 Self-Diagnosis: Greenfield Proposals to Specs ===
test_req0001_sliding_window_allows_within_limit ... ok
test_req0002_rejects_exceeding_limit_and_computes_retry_after ... ok
test_req0003_validates_configuration_and_arguments ... ok
test_req0004_deterministic_clock_expiration_slides_window ... ok
test_req0005_thread_safe_concurrent_requests ... ok
test_req0006_reset_clears_key_state ... ok
-----
Ran 6 tests in 0.001s -- OK

[PASS] All Lab 01 specs, ADRs, Conductor plan, and 6/6 tests verified!
```

## Lab 01 Key Takeaway

- By spending 5 minutes on `/opsx:propose`, 2 ADRs, and a 7-section Conductor `spec.md`, the 60-line implementation and 6 unit tests pass on the very first run with zero ambiguity.

# Lab 02 Step 1: Auditing an Undocumented Brownfield Codebase

Inspecting `labs/lab_02_brownfield_spec_recovery/service/leasemanager.py` via `/opsx:explore`

## The Brownfield Challenge: Code Exists, Specs Don't

- You inherit `service/leasemanager.py` (a working Distributed Lease Lock Manager) and `service/test_leasemanager.py`, with **zero specification files**.
- **Why Direct Agent Edits Are Dangerous:** If you ask an agent to “optimize lease cleanup” without a baseline spec, it might reset the counter when all leases expire—silently breaking monotonic fencing tokens and corrupting downstream storage!

## Inspecting `service/leasemanager.py` in Explore Mode

```
class LeaseLockManager:
    def __init__(self, clock: Callable[[], float] = time.monotonic):
        self._lock = threading.RLock()
        self._leases: dict[str, Lease] = {}
        self._fencing_counter: int = 0 # <- CRITICAL INVARIANT!

    def acquire(self, resource: str, owner: str, ttl_seconds: float):
        ...
        self._fencing_counter += 1

        lease = Lease(resource, owner, self._fencing_counter, now + ttl)
```

## Step 1 Command & Read-Only Investigation Rule

- Run `/opsx:explore` on `labs/lab_02_brownfield_spec_recovery/service/`. Remember: `/opsx:explore` is strictly read-only—it maps every branch, exception, and state transition before writing a single line of specification.

# Lab 02 Step 2: Extracting the 6 Hidden Behavioral Invariants

Mapping every method and guard in `leasemanager.py` to REQ-0001 through REQ-0006

## Invariants REQ-0001 to REQ-0003 (Acquisition & Fencing)

- **REQ-0001 (Exclusive Acquisition):** `acquire(resource, owner, ttl)` grants a lease when the resource has no lease or its existing lease has expired (`now >= expires_at`).
- **REQ-0002 (Monotonic 64-Bit Fencing Token):** Every successful `acquire()` or `renew()` MUST increment `_fencing_counter` strictly monotonically ( $t_{k+1} > t_k$ ), and the counter MUST NEVER reset even when all leases are released or evicted.
- **REQ-0003 (Contended Rejection):** If an active, non-expired lease is held by another owner, `acquire()` MUST return `None`.

## Invariants REQ-0004 to REQ-0006 (Lifecycle & Eviction)

- **REQ-0004 (Active Lease Renewal):** `renew(resource, owner, ttl)` extends `expires_at = now + ttl` and issues a new higher `fencing_token` if and only if `owner` holds an active, non-expired lease.
- **REQ-0005 (Holder-Verified Release):** `release(resource, owner, fencing_token)` removes the lease only when both `owner` and `fencing_token` match the active lease.
- **REQ-0006 (Expired Eviction & Thread Safety):** `evict_expired()` purges expired leases under `threading.RLock` without altering `_fencing_counter`.

## Brownfield Extraction Rule: Capture Essential Behavior, Not Incidental Variable Names

- Notice that REQ-0002 specifies the *observable monotonicity invariant* ( $t_2 > t_1$  across expiration/release cycles) rather than hardcoding private attribute names in the contract.

# Lab 02 Step 3: Why TTL Leases Fail Under GC Pauses ([ADR-0001](#))

Documenting the distributed systems rationale in [expected\\_output/ADR-0001-fencing-tokens.md](#)

## The Split-Brain Zombie Write Hazard (Without Fencing)

1. Worker A acquires lease (`TTL = 5s`) and begins preparing a storage write.
2. Worker A hits a **6-second stop-the-world GC pause** (or network delay).
3. At  $t = 5.1s$ , the lease expires; Worker B acquires the lease and writes valid state to storage.
4. At  $t = 6.0s$ , Worker A wakes up from GC pause and sends its delayed write—**silently overwriting Worker B's state!**

## The Monotonic Fencing Token Solution ([ADR-0001](#))

1. Worker A is issued `fencing_token = 101` upon acquisition.
2. After Worker A's lease expires during its GC pause, Worker B acquires the lease and receives `fencing_token = 102`.
3. Storage records `max_seen_token = 102` when committing Worker B's write.
4. When Worker A wakes up and sends its write with `token = 101 < 102`, storage immediately **rejects the stale write!**

## Why Recovering [ADR-0001-fencing-tokens.md](#) Protects Your Codebase

- By recording this Kleppmann fencing-token proof in [ADR-0001-fencing-tokens.md](#), any engineer or AI agent reading the repo understands immediately why `_fencing_counter` must be global and monotonic.

# Lab 02 Step 4: Authoring `openspec/specs/lease-manager/spec.md` & `design.md`

Writing the normative 4-hashtag Gherkin scenarios and component design doc

## Recovered Spec (`openspec/specs/lease-manager/spec.md`)

```
### Requirement: [REQ-0002] Strictly Monotonic 64-Bit Fencing Tokens
The LeaseLockManager SHALL assign a strictly increasing integer
`fencing_token` ('token_next > token_prev') on every successful
`acquire` or `renew` call. The counter MUST NOT reset when leases
expire or are released.

#### Scenario: Fencing token increments across expiration boundary
- **GIVEN** `"worker-1"` acquires `"db-shard-0"` at `t=0.0` (`ttl=5.0`)
  receiving `fencing_token = 1`
- **WHEN** the clock advances to `t=6.0` and `"worker-2"` acquires
  `"db-shard-0"`
- **THEN** `"worker-2"` SHALL receive `fencing_token = 2` (`2 > 1`).
```

## Recovered Design Document (`expected_output/design.md`)

- **Context:** Documents the in-memory `LeaseLockManager` and immutable `Lease( dataclass(frozen=True) )` value object.
- **Goals / Non-Goals:** Guarantees single-process mutual exclusion and monotonic fencing token issuance; out-of-process Raft consensus is a Non-Goal.
- **Concurrency Invariant:** All mutations to `_leases` and `_fencing_counter` execute atomically inside `with self._lock:`.

## Baseline Spec Promotion

- Once verified, your recovered `spec.md` lives permanently at `openspec/specs/lease-manager/spec.md`. The brownfield subsystem is now upgraded to **SDD Maturity Level 2 (Spec-Anchored)**!

# Lab 02 Step 5: Verifying Zero-Regression Brownfield Recovery

Running `./labs/lab_02_brownfield_spec_recovery/self_diagnose.sh`

## What `lab_02/self_diagnose.sh` Checks

- 1. Normative Spec Completeness:** Confirms `REQ-0001` through `REQ-0006` are present in `expected_output/openspec/specs/lease-manager/spec.md`.
- 2. RFC 2119 & 4-Hashtag Gherkin Compliance:** Confirms uppercase `SHALL/MUST` and `#### Scenario:` blocks with `GIVEN`, `WHEN`, `THEN`.
- 3. ADR & Design Completeness:** Confirms `ADR-0001-fencing-tokens.md` and `design.md` document split-brain fencing protection.
- 4. Brownfield Test Suite Execution:** Runs all 6 unit tests in `service/test_leasemanager.py`.

## Terminal Output (`./self_diagnose.sh`)

```
$ ./labs/lab_02_brownfield_spec_recovery/self_diagnose.sh
=== Lab 02 Self-Diagnosis: Brownfield Spec Recovery ===
test_acquire_and_contended_rejection ... ok
test_expiration_allows_new_owner_with_higher_fencing_token ... ok
test_fencing_token_strictly_monotonic_across_release ... ok
test_holder_verified_release_guards_owner_and_token ... ok
test_renew_extends_ttl_and_increments_token ... ok
test_evict_expired_cleans_only_expired_leases ... ok
-----
Ran 6 tests in 0.000s -- OK
[PASS] Brownfield unit tests and recovered OpenSpec REQ-0001..0006 +
      ADR-0001 verified.
```

## Lab 02 Key Takeaway

- Never unleash a coding agent on legacy code blind. Spend 10 minutes running `/opsx:explore` to recover `spec.md` and `ADR-0001` first—then every future change is anchored to verified invariants.

# Lab 03 Step 1: Inspecting the Approved Circuit Breaker Contract

Walking through `labs/lab_03_from_specs_to_tdd_code/specs/spec.md` (REQ-0001..REQ-0007)

## The 3-State Machine (`CLOSED` → `OPEN` → `HALF_OPEN`)

- **`CLOSED` (Normal):** Calls pass through; consecutive failures increment counter; any success resets counter to 0.
- **`OPEN` (Fast-Fail):** Triggered when consecutive failures reach `failure_threshold`. Rejects calls immediately with `CircuitOpenError` without calling downstream function.
- **`HALF_OPEN` (Probe):** Entered after `recovery_timeout_seconds`. Admits 1 probe call: success → `CLOSED`; failure → `OPEN`.

## The 7 Normative Requirements (`specs/spec.md`)

- **REQ-0001:** Initial `CLOSED` state forwards calls and returns results.
- **REQ-0002:** Reaching `failure_threshold` trips state to `OPEN`.
- **REQ-0003:** `OPEN` state raises `CircuitOpenError` with zero downstream calls.
- **REQ-0004:** Cooldown elapsed transitions `OPEN` → `HALF_OPEN`.
- **REQ-0005:** Successful probe in `HALF_OPEN` resets streak and closes circuit.
- **REQ-0006:** Failed probe in `HALF_OPEN` immediately re-opens circuit.
- **REQ-0007:** Success in `CLOSED` resets prior sub-threshold failure streak.

## Goal-Driven Execution Setup

- In Lab 03, the spec (`specs/spec.md`) and design (`specs/design.md`) are already approved. Your job is to translate `REQ-0001..REQ-0007` into a sequenced TDD task plan and 1-to-1 traceable unit tests.

# Lab 03 Step 2: Decomposing Requirements into Sequenced TDD Tasks

Structuring `expected_output/tasks.md` so every task pairs a `test_reqXXXX_*` test with minimal code

## Sequenced TDD Task Plan (`expected_output/tasks.md`)

```
## Batch 1: Core State Model & Closed/Open Trip (REQ-0001, REQ-0002, REQ-0007)
- [x] 1.1 Write test_req0001_closed_state_forwards_calls (Red) then
    implement CircuitState & CLOSED call forwarding (Green).
- [x] 1.2 Write test_req0002_failure_threshold_trips_to_open (Red) then
    implement failure counter & OPEN transition (Green).
- [x] 1.3 Write test_req0007_success_resets_failure_streak (Red) then
    reset failure count on CLOSED success (Green).

## Batch 2: Fast-Fail & Half-Open Recovery (REQ-0003, REQ-0004, REQ-0005, REQ-0006)
- [x] 2.1 Write test_req0003_open_state_fails_fast_without_invoking_callable.
- [x] 2.2 Write test_req0004_cooldown_transitions_to_half_open.
- [x] 2.3 Write test_req0005_probe_success_closes_circuit &
    test_req0006_probe_failure_reopens_circuit.
```

## Why 1-to-1 Requirement-to-Test Naming Matters

- Naming every test `test_reqXXXX_<behavior>` (and putting `Verifies REQ-XXXX` in the docstring) creates an **auditable bi-directional link** between `spec.md` and your test runner.
- When `REQ-0004` changes in a future OpenSpec delta, the engineer (or agent) knows immediately which unit test must be updated first in the **Red** phase!

## The Strict Red → Green → Refactor Rule (`/conductor:implement`)

- Never allow an agent to write `circuitbreaker.py` first and “backfill” tests afterward. Write the failing test first, run `CI=true python3 -m unittest` to witness the failure, then write the code.

# Lab 03 Step 3: Red-Green-Refactor Batch 1 (CLOSED & OPEN Fast-Fail)

Implementing `REQ-0001`, `REQ-0002`, `REQ-0003`, and `REQ-0007` in `circuitbreaker.py`

## Traceable Unit Tests (`test_circuitbreaker.py`)

```
def test_req0002_failure_threshold_trips_to_open(self) -> None:
    """Verifies REQ-0002: Consecutive failures trip CLOSED -> OPEN."""
    cb = CircuitBreaker(failure_threshold=2, recovery_timeout_seconds=10.0)
    with self.assertRaises(RuntimeError):
        cb.call(self._fail)
    self.assertEqual(cb.state, CircuitState.CLOSED)
    with self.assertRaises(RuntimeError):
        cb.call(self._fail)
    self.assertEqual(cb.state, CircuitState.OPEN)

def test_req0003_open_state_fails_fast_without_invoking_callable(self):
    """Verifies REQ-0003: OPEN rejects with CircuitOpenError (0 calls)."""
    ...
    with self.assertRaises(CircuitOpenError):
        cb.call( tracked_callable )

    self.assertEqual(invocations, 0)
```

## Minimal Production Code (`circuitbreaker.py`)

```
if current_state == CircuitState.OPEN:
    raise CircuitOpenError("Circuit breaker is OPEN; failing fast.")

try:
    result = func(*args, **kwargs)
except Exception:
    with self._lock:
        self._failure_count += 1
        self._last_failure_time = float(self._clock())
        if (self._state == CircuitState.HALF_OPEN
            or self._failure_count >= self.failure_threshold):
            self._state = CircuitState.OPEN
            self._half_open_flight = 0

    raise
```

## Why `test_req0003` Explicitly Asserts `invocations == 0`

- A classic LLM bug is calling `func()` *before* checking if the circuit is `OPEN`. Because the Gherkin scenario in `REQ-0003` states “without invoking the wrapped callable”, our test asserts `invocations == 0`.

# Lab 03 Step 4: Red-Green-Refactor Batch 2 (HALF\_OPEN Probe Recovery)

Implementing `REQ-0004`, `REQ-0005`, and `REQ-0006` with deterministic clock injection

## Cooldown & Single-Probe Gate (`circuitbreaker.py`)

```
def _evaluate_state_unlocked(self) -> CircuitState:
    if self._state == CircuitState.OPEN and self._last_failure_time is not None:
        elapsed = float(self._clock()) - self._last_failure_time
        if elapsed >= self.recovery_timeout_seconds:
            self._state = CircuitState.HALF_OPEN
            self._half_open_in_flight = 0
        return self._state

# Inside call():
if current_state == CircuitState.HALF_OPEN:
    if self._half_open_in_flight >= self.half_open_max_calls:
        raise CircuitOpenError("Half-open probe limit reached")

    self._half_open_in_flight += 1
```

## Probe Success vs. Probe Failure (`REQ-0005` & `REQ-0006`)

- **On Probe Success (`REQ-0005`):** Inside `with self._lock:`, reset `self._failure_count = 0`, `self._half_open_in_flight = 0`, and transition `self._state = CircuitState.CLOSED`.
- **On Probe Failure (`REQ-0006`):** Inside the `except Exception:` handler, record `self._last_failure_time = float(self._clock())` and immediately transition back to `CircuitState.OPEN` for another full cooldown cycle.

## Zero `time.sleep()` in Unit Tests

- Because `CircuitBreaker` accepts an injectable `clock` lambda, `test_req0004`, `test_req0005`, and `test_req0006` advance simulated time from  $t = 0.0$  to  $t = 15.0$  instantaneously—running all 7 tests in **1 millisecond!**

# Lab 03 Step 5: Git Notes Audit Trail & Running `./self_diagnose.sh`

Verifying 100% requirement-to-test traceability across `REQ-0001..REQ-0007`

## Conductor `git notes` & Checkpoint Recording

```
$ git commit -m "feat(circuit-breaker): implement HALF_OPEN probe recovery"
$ git notes add -m "Track: circuit_breaker_20261008"
Tasks: 2.1, 2.2, 2.3 (REQ-0003..REQ-0006)
Tests: 7/7 passing (CI=true python3 -m unittest)" HEAD
# Phase Checkpoint in conductor/tracks/<id>/plan.md:
## Phase 2: Fast-Fail & Half-Open Recovery [checkpoint: 8f9e0a1]
```

- Attaching `git notes` keeps commit titles clean while storing structured task telemetry directly in Git object storage.

## Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_03_from_specs_to_tdd_code/self_diagnose.sh
=== Lab 03 Self-Diagnosis: From Specs to TDD Code (Circuit Breaker) ===
test_req0001_closed_state_forwards_calls ... ok
test_req0002_failure_threshold_trips_to_open ... ok
test_req0003_open_state_fails_fast_without_invoking_callable ... ok
test_req0004_cooldown_transitions_to_half_open ... ok
test_req0005_probe_success_closes_circuit ... ok
test_req0006_probe_failure_reopens_circuit ... ok
test_req0007_success_resets_failure_streak ... ok
-----
Ran 7 tests in 0.001s -- OK

[PASS] All 7 CircuitBreaker requirements (REQ-0001..REQ-0007) verified!
```

## Lab 03 Key Takeaway

- When every requirement ID (`REQ-0001..REQ-0007`) maps 1-to-1 to a Red-Green-Refactor unit test, verification becomes a deterministic  $< 1\text{ms}$  check rather than a subjective human eyeball review.

# Lab 04 Step 1: Detecting Spec-to-Code Drift ([REQ-0005](#) Starvation Bug)

Auditing `labs/lab_04_drift_detection_and_5step_refactoring/service/quota_allocator.py`

## What `specs/spec.md` Requires ([REQ-0005](#))

```
### Requirement: [REQ-0005] Fractional Replenishment Progress Preservation
Calls to 'consume' or 'available' that occur before a full integer
token has accumulated ('elapsed * refill_rate_per_sec < 1.0') MUST
NOT discard fractional elapsed time or starve token replenishment.
```

```
#### Scenario: High-frequency polling does not starve token refill
- **GIVEN** a bucket with 'capacity=5', 'tokens=0', 'refill_rate=1.0/s'
- **WHEN** a client polls 'consume(1)' at 't=0.4s', '0.8s', and '1.2s'
- **THEN** at 't=1.2s' the bucket SHALL have replenished '1' token
```

and admit the request.

## The Bug in `service/quota_allocator.py`

```
def _replenish(self, bucket: _BucketState, now: float) -> None:
    elapsed = max(0.0, now - bucket.last_replenished)
    added = int(elapsed * self._refill_rate_per_sec)
    if added > 0:
        bucket.tokens = min(self._capacity, bucket.tokens + added)
    # DRIFT BUG: unconditionally overwrites timestamp even when added == 0!

    bucket.last_replenished = now
```

- When polled every 0.4s at 1 token/s, `added == int(0.4) == 0`, yet `last_replenished` resets to `now`—**permanently starving the bucket forever!**

## Why Existing Happy-Path Tests Missed This Drift

- Happy-path tests only advanced the clock by whole seconds ( $t = 0 \rightarrow t = 5$ ). Only by auditing the code against [REQ-0005](#)'s sub-quantum Gherkin scenario do we expose the timestamp overwrite bug.

# Lab 04 Step 2: The 5-Step Brownfield Scorecard Refactoring Loop

Triaging labs/lab\_04\_drift\_detection\_and\_5step\_refactoring/scorecard.md (Steps 1 & 2)

## Engineering Audit Scorecard (scorecard.md)

```
| ID | Severity | Finding Description | Decision |
|----|-----|-----|-----|
| F-01 | CRITICAL | REQ-0005 fractional timestamp overwrite | ACCEPTED_FOR_REFACTOR |
| | | starves refill under sub-quantum polling. | |
| F-02 | LOW | Export Prometheus gauge metrics for | DEFERRED (Q4) |
| | | per-tenant token balances. | |
| F-03 | LOW | Migrate in-memory dict to Redis cluster. | DEFERRED |
| F-04 | NIT | Rewrite docstrings to NumPy format. | REJECTED |
```

## The 5-Step Brownfield Refactoring Loop

- Step 1 (Audit):** Run static/spec audit to populate `scorecard.md`.
- Step 2 (Human Go/No-Go Triage):** Lead engineer marks each finding `ACCEPTED_FOR_REFACTOR`, `DEFERRED`, or `REJECTED`.
- Step 3 (Surgical Track Spec):** Create Conductor track for `ACCEPTED` item(s) only; lock `DEFERRED/REJECTED` in `Out of Scope`.
- Step 4 (Red-Green-Refactor TDD):** Write regression test, apply minimal fix.
- Step 5 (Review & Sync):** Run `/conductor:review` & `/opsx:verify`.

## Why Step 2 (Scorecard Triage) Stops “Mega-Prompt” Disasters

- If you feed all 4 findings to an agent at once, it will mix a critical concurrency bugfix (`F-01`) with a Redis rewrite (`F-03`) and docstring churn (`F-04`). The scorecard enforces **one surgical track per critical finding**.

# Lab 04 Step 3: Enforcing Karpathy’s “Surgical Changes” Rule

Locking F-02, F-03, and F-04 inside Section 6 (Out of Scope) of `expected_output/spec.md`

## Surgical Conductor Track `spec.md` (`expected_output/spec.md`)

```
## 1. Overview
Remediate Scorecard Finding F-01 (REQ-0005 fractional replenishment
starvation) in TokenBucketQuotaAllocator._replenish().

## 3. Functional Requirements
- Preserve fractional elapsed time `(elapsed * rate) % 1.0` across
  sub-quantum calls by advancing `last_replenished` by `added / rate`
  only when `added > 0` (or `now` when bucket reaches full capacity).

## 6. Out of Scope
- F-02 (DEFERRED): Prometheus gauge metrics export.
- F-03 (DEFERRED): Distributed Redis backend migration.

- F-04 (REJECTED): NumPy docstring reformatting.
```

## How Out of Scope Acts as a Hard Guardrail

- During `/conductor:implement`, the agent reads `spec.md`. Because F-02, F-03, and F-04 are explicitly listed in `## 6. Out of Scope`, the agent is forbidden from touching docstrings, adding metrics, or importing Redis.
- During `/conductor:review`, if the git diff touches anything in `Out of Scope`, the automated review flags a **Plan Compliance Violation!**

## Result: A 6-Line Surgical Diff Instead of a 500-Line Rewrite

- Human review time drops from 45 minutes to **30 seconds**: the diff only modifies `_replenish()` and adds `test_req0005_sub_quantum_polling_*`.

# Lab 04 Step 4: Writing the Regression Test & Surgical Code Fix

Executing Step 4 (Red → Green) in `quota_allocator_fixed.py` and `test_quota_allocator.py`

## 1. The Red Phase: Failing REQ-0005 Regression Test

```
def test_req0005_sub_quantum_polling_preserves_fractional_elapsed_time(self):
    """Verifies REQ-0005: Sub-quantum polling does not starve refill."""
    now = [0.0]
    alloc = TokenBucketQuotaAllocator(capacity=2, refill_rate_per_sec=1.0,
                                      clock=lambda: now[0])
    self.assertTrue(alloc.consume("tenant-1", 2)) # Drain to 0 at t=0.0
    now[0] = 0.4
    self.assertFalse(alloc.consume("tenant-1", 1)) # 0.4s < 1.0s
    now[0] = 0.8
    self.assertFalse(alloc.consume("tenant-1", 1)) # 0.8s < 1.0s
    now[0] = 1.2
    # At t=1.2s, 1.2s total elapsed -> 1 token MUST be available!

    self.assertTrue(alloc.consume("tenant-1", 1))
```

## 2. The Green Phase: Surgical Fix in `_replenish()`

```
def _replenish(self, bucket: _BucketState, now: float) -> None:
    if now <= bucket.last_replenished:
        return
    elapsed = now - bucket.last_replenished
    added = int(elapsed * self._refill_rate_per_sec)
    if added > 0:
        new_tokens = bucket.tokens + added
        if new_tokens >= self._capacity:
            bucket.tokens = self._capacity
            bucket.last_replenished = now
        else:
            bucket.tokens = new_tokens

        bucket.last_replenished += added / self._refill_rate_per_sec
```

### Mathematical Precision of `bucket.last_replenished += added / rate`

- At  $t = 0.4s$  and  $t = 0.8s$ , `added == 0` so `last_replenished` remains 0.0. At  $t = 1.2s$ , `added == 1`, so `last_replenished` advances to  $0.0 + 1/1.0 = 1.0s$ —preserving the remaining 0.2s fractional credit!

# Lab 04 Step 5: Differential Bug/Fix Verification in `./self_diagnose.sh`

Proving the regression test fails on the bugged service and passes on the fixed service

## How `lab_04/self_diagnose.sh` Proves the Fix

- 1. Negative Control (Bugged Service):** Runs `test_req0005` against `service/quota_allocator.py` and verifies that it **fails** (proving the test genuinely catches the starvation bug).
- 2. Positive Control (Fixed Service):** Runs the full 6-test suite (`REQ-0001..REQ-0006`) against `expected_output/quota_allocator_fixed.py` and verifies **6/6 pass**.
- 3. Scorecard & Scope Audit:** Confirms `expected_output/spec.md` contains all 7 sections and lists `F-02`, `F-03`, and `F-04` under `Out of Scope`.

## Live Verification Output (`./self_diagnose.sh`)

```
$ ./labs/lab_04_drift_detection_and_5step_refactoring/self_diagnose.sh
=== Lab 04 Self-Diagnosis: Spec Drift & 5-Step Refactoring ===
test_req0001_initial_bucket_starts_at_full_capacity ... ok
test_req0002_insufficient_tokens_leaves_balance_unchanged ... ok
test_req0003_refill_caps_at_capacity ... ok
test_req0004_validation_rejects_invalid_inputs ... ok
test_req0005_sub_quantum_polling_preserves_fractional_elapsed_time ... ok
test_req0006_concurrent_consumption_safety ... ok
-----
Ran 6 tests in 0.001s -- OK
[PASS] Drift bug confirmed in service/quota_allocator.py, fixed in
      expected_output/, and scoped via 5-Step Scorecard Track Spec.
```

## Lab 04 Key Takeaway

- A regression test is only proven when you witness it **fail** on the buggy code (**Red**) and **pass** on the surgical fix (**Green**).

# Lab 05 Step 1: Auditing an Incoming “Vibe-Coded” Pull Request

Inspecting `labs/lab_05_sdd_code_review_and_stacked_prs/simulated_review/subpar_change.patch`

## Baseline System Truth (`base/openspec/specs/kv-cache/spec.md`)

- **REQ-0001:** Bounded key-value storage up to `max_size`.
- **REQ-0002 (FIFO Eviction Invariant):** When inserting a new key at capacity, `KVCache` MUST evict the oldest inserted key in **First-In-First-Out (FIFO)** order. Reading a key via `get(key)` MUST NOT alter eviction order!
- **REQ-0003:** Updating an existing key preserves its FIFO position.
- **REQ-0004 & REQ-0005:** Explicit `delete(key)` & input validation.

## The Incoming Patch (`simulated_review/subpar_change.patch`)

```
@@ -28,6 +28,7 @@ class KVCache:
def get(self, key: str) -> Optional[ str ]:
if key not in self._store:
return None
+ self._store.move_to_end(key) # <- "Optimized cache hits!"
return self._store[key]

+def put_with_ttl(self, key: str, val: str, ttl: float) -> None:
+ self._store[key] = val

+ self._expiry[key] = time.time() + ttl
```

## Why Traditional “Code-Only” Review Misses This

- A reviewer looking only at Python syntax thinks `self._store.move_to_end(key)` looks clean! Only by reviewing against `openspec/specs/kv-cache/spec.md` do you catch that **REQ-0002** mandates strict **FIFO** order.

# Lab 05 Step 2: Catching All 3 SDD Violations via [/conductor:review](#)

How Spec-Driven Code Review blocks behavioral drift, un-specced APIs, and untested code

## Violation 1: Spec Contradiction

Breaks [REQ-0002](#) (FIFO → LRU)

- Calling `move_to_end(key)` inside `get()` mutates FIFO eviction into Least-Recently-Used (LRU) eviction.
- Downstream callers relying on predictable FIFO cohort expiration suffer silent data bugs.

## Violation 2: Un-Specced API

Zero Spec Delta for `put_with_ttl`

- Adds public method `put_with_ttl(key, value, ttl_seconds)` with no [REQ-0006](#) in [openspec/specs/kv-cache/spec.md](#).
- Fails to purge expired entries before evicting valid FIFO keys at capacity!

## Violation 3: Untested & Non-Hermetic

Hardcoded `time.time()` & 0 Tests

- Uses non-injectable `time.time()` instead of deterministic `clock`.
- Fails to validate `ttl_seconds <= 0` and includes zero `test_req0006_*` unit tests.

## The SDD Code Review Golden Rule

- **Review the Spec Diff First:** Every PR that adds or changes observable behavior MUST include an OpenSpec delta ([REQ-XXXX](#)) and a matching `test_reqXXXX_*` test. If code and spec disagree, block the PR.

# Lab 05 Step 3: Synchronizing the OpenSpec Contract (REQ-0006)

Adding REQ-0006 to `expected_output/openspec/specs/kv-cache/spec.md` while preserving REQ-0002

## Synchronized Spec Delta (REQ-0006: Per-Entry TTL Expiration)

```
### Requirement: [REQ-0006] Optional Per-Entry TTL & Capacity Reclamation
The KVCache SHALL support 'put_with_ttl(key, value, ttl_seconds)'
where 'ttl_seconds > 0'. Expired entries ('now >= expires_at') MUST
be treated as absent on 'get(key)' and MUST be purged before evicting
any non-expired FIFO entry when inserting at capacity.

#### Scenario: Expired entry is purged before FIFO eviction at capacity
- **GIVEN** a KVCache('max_size=2') with '"k1"' ('ttl=5.0') and '"k2"'
- **WHEN** clock advances to 't=6.0' and '"k3"' is inserted via 'put'
- **THEN** expired '"k1"' SHALL be purged, '"k2"' SHALL remain present,
  and '"k3"' SHALL be stored without evicting '"k2"'.
```

## Why Spec Writing Exposed a Subtle Capacity Bug

- Notice the scenario above: when the cache is full (`max_size=2`) and `"k1"` has expired, inserting `"k3"` should reclaim `"k1"`'s slot—**not** evict a live key!
- The vibe-coded patch forgot to call `_purge_expired()` inside `put()` before checking `len(self._store) >= self.max_size`. Writing REQ-0006 caught that edge case before production!

## Preserving REQ-0002 Intact

- In `expected_output/openspec/specs/kv-cache/spec.md`, REQ-0001 through REQ-0005 remain intact (strict FIFO order), and REQ-0006 is cleanly appended.

# Lab 05 Step 4: Remediating `kvcache.py` & Verifying `test_kvcache.py`

Fixing FIFO preservation, eager+lazy TTL expiration, and deterministic clock injection

## Compliant Implementation (`expected_output/kvcache.py`)

```
def _purge_expired_unlocked(self, now: float) -> None:
    expired = [k for k, exp in self._expiry.items() if now >= exp]
    for k in expired:
        self._store.pop(k, None)
        self._expiry.pop(k, None)

def get(self, key: str) -> Optional[str]:
    with self._lock:
        now = float(self._clock())
        if key in self._expiry and now >= self._expiry[key]:
            self._store.pop(key, None); self._expiry.pop(key, None)
            return None

    return self._store.get(key) # <- Preserves REQ-0002 FIFO order!
```

## Traceable Test Suite (`expected_output/test_kvcache.py`)

```
test_req0001_put_and_get_roundtrip ... ok
test_req0002_fifo_eviction_unaffected_by_get_reads ... ok
test_req0003_in_place_update_preserves_fifo_order ... ok
test_req0004_explicit_key_deletion ... ok
test_req0005_validation_rejects_invalid_size_or_key ... ok

test_req0006_per_entry_ttl_expiration_and_capacity_reclamation ... ok
```

- `test_req0002` explicitly reads the oldest key via `get("k1")` before inserting `"k3"` at capacity and asserts `"k1"` is still evicted first (blocking any future LRU regression).

## Eager + Lazy TTL Reclamation

- `put()` and `put_with_ttl()` call `_purge_expired_unlocked(now)` *before* checking capacity so expired keys never cause live FIFO keys to be evicted prematurely.

# Lab 05 Step 5: Stacked Multi-PR Decomposition & Archive PR <sup>N+1</sup>

Structuring `expected_output/tasks.md` for clean Git / Jujutsu (`jj`) stacked pull requests

## Stacked PR Task Breakdown (`expected_output/tasks.md`)

```
## Batch 1: Spec Delta & Core TTL State (PR 1)
- [x] 1.1 Add REQ-0006 to openspec/specs/kv-cache/spec.md & design.md
- [x] 1.2 Add injectable clock & _expiry dict to KVCache

## Batch 2: TTL Eviction, FIFO Remediation & Tests (PR 2)
- [x] 2.1 Implement put_with_ttl() & _purge_expired_unlocked()
- [x] 2.2 Remove move_to_end() from get() to restore REQ-0002 FIFO
- [x] 2.3 Add test_req0006_per_entry_ttl_expiration_and_capacity_reclamation

## Batch 3: Dedicated OpenSpec Archive (PR N+1 -- Docs Only)
- [x] 3.1 Run /opsx:verify & /opsx:archive after PR 1 & PR 2 merge
```

## The 4 Stacked Multi-PR Rules

- 1. One Change Folder = One Feature:** Keep a single `openspec/changes/<id>/` folder across the stack.
- 2. Batch `tasks.md` by PR:** Group checkboxes under `Batch 1 (PR 1)`, `Batch 2 (PR 2)`.
- 3. Truthful Same-PR Checkmarks:** Only mark `[x]` for tasks whose code + tests are in that PR.
- 4. Dedicated Archive PR `N+1`:** Never run `/opsx:archive` mid-stack (moving files to `archive/` breaks downstream rebase!). Run `/opsx:archive` in a final docs-only PR `N+1`.

## Lab 05 Verification Command

- Run `./labs/lab_05_sdd_code_review_and_stacked_prs/self_diagnose.sh` to verify the `REQ-0006` contract, Stacked PR `tasks.md` batches, and all 6 `KVCache` unit tests.

# Lab 06 Step 1: The 8-Section SPEC.md (Sections 1–4)

Inspecting `labs/lab_06_capstone_ssot_and_rebuild_test/SPEC.md` for `CoursePulse`

## Sections 1 & 2: Domain Intent & Architecture

- **Section 1 (System Overview & Domain Intent):** `CoursePulse` manages university course creation, seat-bounded student enrollment, idempotent duplicate checks, and automatic FIFO waitlist promotion when an enrolled student drops.
- **Section 2 (Architecture & Concurrency):** Thread-safe in-memory domain service guarded by `threading.RLock` with zero external runtime dependencies.

## Sections 3 & 4: Data Schemas & API Contracts

- **Section 3 (Data Models):** Immutable `EnrollmentResult(course_id, student_id, status, waitlist_position)` and `DropResult(course_id, dropped_student_id, was_enrolled, promoted_student_id)` dataclasses.
- **Section 4 (Interface Contracts):** Exact signatures and error semantics for `create_course`, `enroll`, `drop`, and `get_roster` (raising `KeyError` on unknown `course_id` and `ValueError` on `capacity < 1`).

## Essential vs. Incidental Complexity in SPEC.md

- Sections 1–4 specify every **observable type**, **field name**, **status string** ("`ENROLLED`", "`WAITLISTED`"), and **exception type** so an independent test suite can bind to the regenerated code with zero glue adjustments.

# Lab 06 Step 2: The 8-Section SPEC.md (Sections 5–8)

State machines, parameterized config, and the 3-Phase Clean-Room Rebuild Recipe

## Sections 5, 6 & 7: Workflows, Layout & Config

- **Section 5 (Behavioral Workflows & REQ-0001..0006):**
  - REQ-0002: Seat available → ENROLLED; full → WAITLISTED (1-based position).
  - REQ-0003: Duplicate `enroll()` is idempotent (no duplicate seat/waitlist entry).
  - REQ-0004: Dropping an ENROLLED student immediately promotes head of FIFO waitlist (`popleft()`) into the vacated seat!
- **Sections 6 & 7:** Module layout (`coursepulse.py`) & env parameterization (`${PORT:-8080}`).

## Section 8: The 3-Phase Clean-Room Rebuild Recipe

**Phase A (Data Models & Validation):** Generate `EnrollmentResult`, `DropResult`, `CourseRoster`, and input validators (REQ-0001).

**Phase B (Core Enrollment & FIFO Waitlist Engine):** Implement `enroll()`, `drop()` with atomic FIFO head promotion, and defensive copy `get_roster()` (REQ-0002..0006).

**Phase C (Adversarial Verification):** Execute read-only `test_rebuild_contract.py` in an isolated `/tmp/` clean room.

## The Level-3 SSOT Litmus Test

- If an engineer or AI agent can read `SPEC.md` in an empty directory and produce `coursepulse.py` that passes all 6 adversarial contract tests on the first run, `SPEC.md` is a true **Single Source of Truth**.

# Lab 06 Step 3: Clean-Room Rebuild & `chmod -w` Adversarial Isolation

How `labs/lab_06_capstone_ssot_and_rebuild_test/self_diagnose.sh` enforces Karpathy's Immutable Verifier

## Clean-Room Rebuild Harness (`lab_06/self_diagnose.sh`)

```
# 1. Create isolated clean-room workspace in /tmp
CLEAN_ROOM="$(mktemp -d /tmp/sdd_rebuild_XXXXXX)"
trap 'chmod -R u+w "$CLEAN_ROOM" 2>/dev/null; rm -rf "$CLEAN_ROOM"' EXIT

# 2. Copy SPEC.md, regenerated implementation, and adversarial tests
cp "$SCRIPT_DIR/SPEC.md" "$CLEAN_ROOM/SPEC.md"
cp "$SCRIPT_DIR/expected_output/coursepulse.py" "$CLEAN_ROOM/coursepulse.py"
mkdir -p "$CLEAN_ROOM/tests"
cp "$SCRIPT_DIR/adversarial_tests/test_rebuild_contract.py"
  "$CLEAN_ROOM/tests/test_rebuild_contract.py"

# 3. Lock adversarial test directory READ-ONLY (chmod -w)

chmod -R a-w "$CLEAN_ROOM/tests"
```

## Why `chmod -R a-w` Is Critical for Agentic Verification

- Just like Karpathy's `autoresearch` locks `prepare.py` (`val_bpb` evaluator) out of the agent's write scope, locking `tests/` with `chmod -R a-w` physically prevents an implementation agent from editing or deleting test assertions.
- We also separate roles: **Agent A (Test Scaffolder)** writes `test_rebuild_contract.py` from `SPEC.md`, while **Agent B (Implementer)** writes `coursepulse.py` without permission to modify `tests/`.

## Zero Shared State with the Original Repo

- Running `python3 -m unittest` inside `$CLEAN_ROOM` guarantees `coursepulse.py` doesn't rely on hidden files or undeclared imports in the parent repository.

# Lab 06 Step 4: Inspecting `coursepulse.py` & `test_rebuild_contract.py`

Verifying FIFO waitlist promotion (`REQ-0004`) and defensive roster snapshots (`REQ-0006`)

## Atomic Drop & FIFO Waitlist Promotion (`coursepulse.py`)

```
def drop(self, course_id: str, student_id: str) -> DropResult:
    with self._lock:
        course = self._get_course_unlocked(course_id)
        if student_id in course.enrolled:
            course.enrolled.remove(student_id)
            promoted: Optional[str] = None
            while course.waitlist:
                candidate = course.waitlist.popleft()
                if candidate in course.waitlisted_set:
                    course.waitlisted_set.remove(candidate)
                    course.enrolled.append(candidate)
                    promoted = candidate
                    break
            return DropResult(course_id, student_id, True, promoted)
```

## Live Clean-Room Verification (`./self_diagnose.sh`)

```
$ ./labs/lab_06_capstone_ssot_and_rebuild_test/self_diagnose.sh
=== Lab 06 Self-Diagnosis: 8-Section SPEC.md & Rebuild Test ===
test_req0001_course_creation_and_validation ... ok
test_req0002_seat_allocation_and_waitlist_overflow ... ok
test_req0003_idempotent_duplicate_enrollment ... ok
test_req0004_fifo_waitlist_promotion_on_enrolled_drop ... ok
test_req0005_waitlist_drop_and_non_member_drop ... ok
test_req0006_immutable_roster_snapshot_and_missing_course_errors ... ok
-----
Ran 6 tests in 0.001s -- OK
[PASS] Clean-Room Rebuild Test passed all read-only adversarial

        contract tests (REQ-0001..REQ-0006).
```

## Defensive Roster Snapshot (`REQ-0006`)

- `test_req0006` mutates the tuple/list returned by `get_roster()` and verifies that internal course state is completely unaffected—proving deep immutability across the boundary.

# Master Diagnostic Checklist & Running `./self_diagnose_all.sh`

Your complete toolkit for teaching and practicing Spec-Driven Development

## Top 4 Practitioner Traps to Avoid

- 1. **Partial `MODIFIED` Block in OpenSpec:** `MODIFIED` replaces the *entire* requirement block on archive. Always copy the full requirement + all existing scenarios before editing!
- 2. **3-Hashtag `### Scenario: Header:`** OpenSpec's parser strictly requires **4 hash-tags** (`#### Scenario:`); 3 hashtags silently drop scenarios.
- 3. **`RENAMED` + `MODIFIED` Header Collision:** `MODIFIED` must reference the **new (To)** requirement name.
- 4. **Archiving Mid-Stack:** Only run `/opsx:archive` in final PR `N+1`.

## Run the Full 6-Lab Self-Diagnosis Harness

```
$ git clone https://github.com/adkaspar-google/sdd-course.git
$ cd sdd-course && ./self_diagnose_all.sh

[PASS] Lab 01: Greenfield Rate Limiter (6/6 tests)
[PASS] Lab 02: Brownfield Lease Lock Manager (6/6 tests)
[PASS] Lab 03: Circuit Breaker TDD State Machine (7/7 tests)
[PASS] Lab 04: Token Bucket Drift & Scorecard Refactoring (6/6 tests)
[PASS] Lab 05: KVCache SDD Code Review & Stacked PRs (6/6 tests)
[PASS] Lab 06: CoursePulse SPEC.md Clean-Room Rebuild (6/6 tests)
=====

[ALL PASSED] 6/6 Hands-On Labs & 37/37 Traceable Tests!
```

## Congratulations on Completing `SDD-Crash-Course!`

- **Course Materials in This Repo:** 6 Executable Labs (`labs/`), 16-Page Practical Coursebook PDF (`Coursebook_SDD_Conductor_OpenSpec.pdf`), Dual-Engine Playbook (`playbooks/`), `@writing-arch-specs` Skill (`.agents/skills/`), and 7 Step-by-Step Video Modules (`slides/`).